Roll No: _____ **Date:** _____

Experiment-9
Synchronization

Synchronization is the process of coordinating the execution of multiple processes or threads so that they share resources (like variables, files, or memory) safely without conflicts. It ensures that only one process or thread accesses a shared resource at a time to maintain data consistency and avoid race conditions.

Issues in Synchronization

1. **Race Condition:** Occurs when multiple processes access and modify shared data at the same time, leading to unpredictable results.
2. **Deadlock:** A situation where two or more processes wait indefinitely for each other to release resources.
3. **Starvation:** A process never gets the required resource because others keep getting priority.
4. **Priority Inversion:** A lower-priority process holds a resource needed by a higher-priority process, causing delays.
5. **Busy Waiting (Spinlock):** A process keeps checking for a resource in a loop, wasting CPU time.

Advantages of Synchronization

- **Data Consistency:** Ensures accurate results when multiple threads share data.
- **Prevents Race Conditions:** Avoids incorrect outcomes due to simultaneous access.
- **Coordinated Execution:** Maintains proper order among dependent processes.
- **Safe Resource Sharing:** Prevents data corruption and resource misuse.

Disadvantages of Synchronization

- **Reduced Performance:** Synchronization can slow down execution due to waiting.
- **Risk of Deadlocks:** Improper use of synchronization can cause system hangs.
- **Complex Implementation:** Managing synchronized code adds programming difficulty.
- **CPU Overhead:** Locking, unlocking, and context switching consume CPU time.

Roll No: _____ **Date:** _____

Classical Synchronization Problems and Solutions

1. **Producer-Consumer Problem:** A producer adds items to a buffer and a consumer removes them.
    - *Solution:* Use semaphores to ensure mutual exclusion and prevent buffer overflow/underflow.
2. **Readers-Writers Problem:** Multiple readers can read simultaneously, but only one writer can write at a time.
    - *Solution:* Use semaphores to allow concurrent reading and exclusive writing.
3. **Dining Philosophers Problem:** Philosophers share chopsticks and need two to eat.
    - *Solution:* Use semaphores or resource ordering to prevent deadlock and starvation.
4. **Sleeping Barber Problem:** A barber sleeps if no customers; customers wait if barber is busy.
    - *Solution:* Use semaphores to coordinate barber and customer actions efficiently.

Roll No: _____ **Date:** _____

Dining Philosophers Problem

The Dining Philosophers Problem is a classic synchronization problem where philosophers sit around a table with one chopstick between each pair. Each philosopher needs two chopsticks to eat. The challenge is to coordinate access to the chopsticks so that deadlock and starvation are avoided while allowing all philosophers to eat.

Solution: Use semaphores or resource ordering to ensure mutual exclusion and proper synchronization.

- **Odd Number of Philosophers:** Each philosopher picks the left chopstick first, then the right. Deadlock is unlikely since not all compete for the same chopsticks simultaneously.
- **Even Number of Philosophers:** To prevent deadlock, odd philosophers pick left first and even philosophers pick right first, breaking the circular wait.

Aim: Implementation of Dining Philosophers Problem.

Algorithm

1. **Initialize:** Create `NUM_PHILOSOPHERS` semaphores for chopsticks, each initialized to 1.
2. **For each philosopher thread:** Repeat `EAT_COUNT` times:
   (a) Think for a while.
   (b) Pick up chopsticks:
      - If philosopher is the last (odd case), pick right first, then left.
      - Otherwise, pick left first, then right.
   (c) Eat for a while.
   (d) Put down both chopsticks.
   (e) Think again.
3. **End Thread:** After eating `EAT_COUNT` times, philosopher leaves the table.
4. **Main Thread:**
   - Create philosopher threads.
   - Wait for all threads to finish.
   - Destroy all semaphores.
   - Exit program.

Source Code

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

#define NUM_PHILOSOPHERS 5
#define EAT_COUNT 3 // each philosopher eats 3 times before exiting

sem_t chopsticks[NUM_PHILOSOPHERS];

void* philosopher(void* num) {
    int id = *(int*)num;
    for (int i = 0; i < EAT_COUNT; i++) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        if (id == NUM_PHILOSOPHERS - 1) {
            sem_wait(&chopsticks[(id + 1) % NUM_PHILOSOPHERS]);
            sem_wait(&chopsticks[id]);
        } else {
            sem_wait(&chopsticks[id]);
            sem_wait(&chopsticks[(id + 1) % NUM_PHILOSOPHERS]);
        }

        printf("Philosopher %d is eating (Round %d)\n", id, i + 1);
        sleep(2);

        sem_post(&chopsticks[id]);
        sem_post(&chopsticks[(id + 1) % NUM_PHILOSOPHERS]);

        printf("Philosopher %d finished eating and put down chopsticks\
    n", id);
        sleep(1);
    }

    printf("Philosopher %d is done eating and leaves the table\n", id);
    return NULL;
}

int main() {
    pthread_t philosophers[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        sem_init(&chopsticks[i], 0, 1);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
```

```
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        sem_destroy(&chopsticks[i]);
    }

    printf("\nAll philosophers are done. Program exiting gracefully...\n");
    return 0;
}
```

Output

```
student@cselab:~$ gcc dining_philosopher.c -lpthread
student@cselab:~$ ./a.out
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 0 is eating (Round 1)
Philosopher 2 is eating (Round 1)
Philosopher 0 finished eating and put down chopsticks
Philosopher 2 finished eating and put down chopsticks
Philosopher 4 is eating (Round 1)
Philosopher 1 is eating (Round 1)
Philosopher 0 is thinking
Philosopher 2 is thinking
Philosopher 4 finished eating and put down chopsticks
Philosopher 1 finished eating and put down chopsticks
Philosopher 3 is eating (Round 1)
Philosopher 0 is eating (Round 2)
Philosopher 4 is thinking
Philosopher 1 is thinking
Philosopher 3 finished eating and put down chopsticks
Philosopher 0 finished eating and put down chopsticks
Philosopher 2 is eating (Round 2)
Philosopher 4 is eating (Round 2)
Philosopher 3 is thinking
Philosopher 0 is thinking
Philosopher 2 finished eating and put down chopsticks
Philosopher 4 finished eating and put down chopsticks
Philosopher 1 is eating (Round 2)
Philosopher 3 is eating (Round 2)
Philosopher 2 is thinking
Philosopher 4 is thinking
Philosopher 1 finished eating and put down chopsticks
Philosopher 3 finished eating and put down chopsticks
Philosopher 0 is eating (Round 3)
Philosopher 2 is eating (Round 3)
Philosopher 1 is thinking
Philosopher 3 is thinking
Philosopher 0 finished eating and put down chopsticks
Philosopher 2 finished eating and put down chopsticks
Philosopher 4 is eating (Round 3)
Philosopher 1 is eating (Round 3)
Philosopher 0 is done eating and leaves the table
Philosopher 2 is done eating and leaves the table
```

```
Philosopher 4 finished eating and put down chopsticks
Philosopher 1 finished eating and put down chopsticks
Philosopher 3 is eating (Round 3)
Philosopher 4 is done eating and leaves the table
Philosopher 1 is done eating and leaves the table
Philosopher 3 finished eating and put down chopsticks
Philosopher 3 is done eating and leaves the table

All philosophers are done. Program exiting gracefully...
```